

Классификация нагрузки

Table of contents

Классификация нагрузки	1
Типы операций	1
Метрики нагрузки	2
Характеристики нагрузки	3
Связь метрик: от DAU к QPS	3
Иерархия памяти	3
Оценка производительности: Little's Law	4
Фундаментальные компромиссы	4

Классификация нагрузки

Прежде чем выбирать базу данных или структуру хранения, нужно понять **какая нагрузка** будет на систему. База данных для интернет-магазина (миллионы мелких транзакций) и база данных для аналитики (сканирование миллиардов строк) требуют совершенно разных подходов.

Эта лекция вводит классификацию нагрузок: - **OLTP** (Online Transaction Processing) — много коротких транзакций, чтение/запись отдельных строк (интернет-магазин, банковские переводы, бронирование билетов) - **OLAP** (Online Analytical Processing) — редкие, но тяжелые запросы, сканирование миллионов строк для агрегации (отчёты, бизнес-аналитика, data science)

Для OLTP критично время отклика одной транзакции (миллисекунды), для OLAP — пропускная способность (миллионы строк в секунду). Отсюда разные структуры хранения: OLTP использует строковое хранение (B-tree), OLAP — колоночное (Parquet, ClickHouse).

Лекция показывает, как оценить нагрузку через метрики (RPS, размер данных, паттерны доступа) и на основе этого выбрать подходящее хранилище.

Типы операций

Зачастую взаимодействие с хранилищем сводится к ограниченному набору операций:

Тип	Операция	Пример
Чтение	Point lookup	<code>SELECT * FROM users WHERE id = 123</code>
Чтение	Multi-get	<code>SELECT * FROM users WHERE id IN (1, 2, 3)</code>

Тип	Операция	Пример
Чтение	Range scan	<code>SELECT * FROM orders WHERE date BETWEEN '...' AND '...'</code>
Чтение	Prefix scan	<code>SELECT * FROM logs WHERE key LIKE 'user:123:%'</code>
Чтение	Full scan	<code>SELECT AVG(price) FROM products</code>
Запись	Insert	вставка новой записи (ошибка при дубликate)
Запись	Update	изменение существующей записи
Запись	Upsert	<code>INSERT ... ON CONFLICT DO UPDATE</code>
Запись	Delete	удаление записи
Запись	Batch write	запись нескольких записей
Комб.	Read-modify-write	<code>UPDATE accounts SET balance = balance + 100</code>
Комб.	Increment	<code>INCR views:page:123</code> (атомарный счётчик)

Метрики нагрузки

Метрика	Расшифровка	Описание
DAU	Daily Active Users	уникальные пользователи за сутки
MAU	Monthly Active Users	уникальные пользователи за месяц
RPS	Requests Per Second	пользовательских запросов (например HTTP) в секунду
QPS	Queries Per Second	запросов к БД в секунду
TPS	Transactions Per Second	транзакций БД в секунду
IOPS	I/O Operations Per Second	операций ввода-вывода в секунду

Характеристики нагрузки

Read/Write ratio: - 90/10 — преобладание чтения: кэширование, CDN, статический контент - 50/50 — сбалансированная: социальные сети, интернет-магазины - 10/90 — преобладание записи: логирование, IoT, метрики

Размер записи: - < 1 KB — Метаданные, Сессия, счётчики - 1-100 KB — JSON-документы, профили пользователей - 100 KB - 1 MB — статьи, посты с изображениями - > 1 MB — медиафайлы, бинарные данные

Паттерн доступа: - random — point lookup по ключу, случайные запросы - sequential — последовательное чтение, временные ряды, логи

Требования к латентности: - p50 < 50 мс — медиана, типичный пользовательский опыт - p99 < 200 мс — 1% худших запросов, важно для SLA - p999 < 1 с — 0.1% худших, критично для финтеха

Почему p99 важнее среднего: если p99 = 500 мс при 10000 QPS, то 100 запросов в секунду будут медленными — это 8.6 млн медленных запросов в сутки.

Связь метрик: от DAU к QPS

Ключевая задача — перевести бизнес-метрики (DAU) в технические (QPS, IOPS).

Полезные константы: секунд в сутках $\approx 10^5$ (точно: 86400), в месяце $\approx 2.5 \times 10^6$, в году $\approx 3 \times 10^7$.

Формула DAU $\rightarrow$ RPS:

$$RPS_{\text{средний}} = \frac{DAU \times \text{действий на пользователя}}{10^5}$$

Пример: 100K DAU, 20 действий/день $\rightarrow 100000 \times 20 / 100000 = 20$ RPS (средний).

Учёт пиков: нагрузка распределена неравномерно. Активные часы — 12 из 24, пиковые — 4 часа с нагрузкой в 3× от среднего.

$$RPS_{\text{пиковый}} \approx RPS_{\text{средний}} \times 3$$

RPS $\rightarrow$ QPS: один HTTP-запрос может порождать несколько запросов к БД. - GET / products/:id $\rightarrow$ 1 SELECT (QPS = RPS) - POST /orders $\rightarrow$ BEGIN + 3 INSERT + 2 UPDATE + COMMIT (QPS $\approx 6 \times$ RPS)

Иерархия памяти

Уровень	Латентность	Пропускная способность
L1 cache	~1 нс	~1 ТБ/с
L2 cache	~4 нс	~500 ГБ/с
L3 cache	~12 нс	~200 ГБ/с
RAM	~100 нс	~50 ГБ/с

Уровень	Латентность	Пропускная способность
SSD (random)	~100 мкс	~500 МБ/с
SSD (seq)	~50 мкс	~3 ГБ/с
HDD (random)	~10 мс	~1 МБ/с
HDD (seq)	~5 мс	~200 МБ/с

Случайный доступ на HDD в 200 раз дороже последовательного. На SSD разница меньше (~5-10х), но всё ещё значительна.

Источник: таблица основана на данных Jeff Dean (Google) и Peter Norvig, актуализированных Colin Scott. См. Latency Numbers Every Programmer Should Know и Interactive Latency.

Оценка производительности: Little's Law

Как связать время запроса с пропускной способностью? **Little's Law** — фундаментальный закон теории очередей:

$$L = \lambda \times W$$

Где L — среднее число запросов в системе (concurrency), λ — throughput (RPS), W — среднее время обработки.

Применение для планирования пропускной способности: - Знаем W (время запроса) и L (число ядер) $\rightarrow$ находим λ (max RPS) - Знаем λ (целевой RPS) и L $\rightarrow$ находим требуемое W

Пример: сервер с 28 ядрами, время запроса 1 мс:

$$\lambda = L/W = 28/0.001 = 28000 \text{ RPS (теоретический максимум)}$$

Реальность: накладные расходы (context switching, lock contention, GC) снижают эффективность. Коэффициент полезной работы — 0.3–0.5:

$$\lambda_{\text{реальный}} = 28000 \times 0.4 \approx 11000 \text{ RPS}$$

Фундаментальные компромиссы

Read vs Write: оптимизация под чтение замедляет запись и наоборот.

Space vs Time: - **Компрессия** — тратим CPU за уменьшение места на диске - **Денормализация** — тратим место за скорость чтения - **Индексы** — тратим место и замедляем запись за ускорение чтения